

rebranding_age
nt

Rebranding Agent

Monthly multi-agent visual rebrand pipeline: Ideation → Coding → Review → Test → Publish

Overview

The Rebranding Agent refreshes the Meridian platform's visual identity every month. It researches current world events and mood, generates a harmonious colour palette and typography scheme, applies the changes to 5 frontend files, validates SEO/ADA compliance, verifies the build, and commits & pushes to GitHub — all autonomously. It is built with the OpenAI Agents SDK (openai-agents) and uses agent handoffs to pass control between 5 specialised sub-agents.

Schedule & Invocation

GitHub Actions	.github/workflows/run-rebranding-agent.yml
CLI	python -m meridian_agents.rebranding_agent
Force run	FORCE_REBRAND=true python -m meridian_agents.rebranding_agent
SDK	openai-agents (Runner + Agent with handoffs)
Primary model	gemini-2.5-flash (GOOGLE_API_KEY / GEMINI_API_KEY)
Fallback model	gpt-4o-mini (OPENAI_API_KEY)
Max turns	60 total across the full pipeline
Context object	RebrandCtx dataclass — shared mutable state passed through all agents

Multi-Agent Architecture

Agents are wired with explicit handoff references. Control flows forward through the pipeline and loops back on failures:

```
IdeationAgent → CodingAgent → ReviewerAgent → TesterAgent → PublisherAgent
ReviewerAgent (contrast issues) → IdeationAgent (revise plan) → CodingAgent
ReviewerAgent (code issues) → CodingAgent (targeted fix) → ReviewerAgent
TesterAgent (build/test failure) → CodingAgent (fix) → TesterAgent
```

Sub-Agents

IdeationAgent

Creative director. (1) `check_schedule()` — skips if not time for monthly rebrand. (2) `research_world_events()` — Gemini Search grounding for current month's dominant theme. (3) `generate_rebrand_plan()` — produces Layout A (light) + Layout B (dark) colour schemes, typography, and per-file patch instructions. On revision: `revise_rebrand_plan(feedback)`.

Tools: `check_schedule`, `research_world_events`, `generate_rebrand_plan`, `revise_rebrand_plan`

CodingAgent

Applies plans to frontend files. Fresh run: `patch_frontend_files()` for all 5 files. Targeted fix: `read_frontend_file()` + `write_frontend_file()` for specific issues. Never calls `patch_frontend_files()` during targeted fixes (would overwrite everything).

Tools: `patch_frontend_files`, `read_frontend_file`, `write_frontend_file`

ReviewerAgent

Calls `review_seo_ada()` exactly once per visit. PASSED → hand to TesterAgent. FAILED with contrast issues → back to IdeationAgent. FAILED with code issues → back to CodingAgent. Caps at 3 revision cycles.

Tools: `review_seo_ada`

TesterAgent

`verify_build()` first, then `run_structural_tests()`. Both pass → PublisherAgent. Either fails → CodingAgent with full error. Max one round-trip per issue type.

Tools: `verify_build`, `run_structural_tests`

PublisherAgent

Calls `commit_and_push()`. Reports 'Pushed successfully', 'Nothing to commit', or the error. Does not call any other tools or modify any files.

Tools: `commit_and_push`

Tools Reference

Tool	Purpose
<code>check_schedule()</code>	Reads <code>.last_rebrand</code> file and <code>FORCE_REBRAND</code> flag. Returns <code>SKIP</code> or <code>PROCEED</code> .
<code>research_world_events()</code>	Gemini 2.5 Flash with Google Search grounding — identifies month's theme and mood
<code>generate_rebrand_plan()</code>	LLM call producing colour tokens, typography, and per-file patch plans for A+B layouts
<code>revise_rebrand_plan(feedback)</code>	Re-calls LLM with contrast failure details to produce corrected colour choices
<code>patch_frontend_files()</code>	Applies the full rebrand plan to all 5 allowed frontend files at once
<code>read_frontend_file(name)</code>	Reads a file from the allowed set (<code>style.css</code> , <code>Navbar.vue</code> , <code>Home.vue</code> , <code>Footer.vue</code> , <code>index.html</code>)
<code>write_frontend_file(name, content)</code>	Writes corrected content for targeted fixes

Tool	Purpose
review_seo_ada()	Checks colour contrast (WCAG AA $\geq 4.5:1$), alt text, ARIA labels, marker syntax, SEO meta
verify_build()	npm run build in frontend/ — returns BUILD PASSED or BUILD FAILED with stderr
run_structural_tests()	Checks Vue template structure, Tailwind class validity, marker completeness
revert_changes()	git checkout HEAD -- to undo all changes if needed
commit_and_push()	git add -u + git commit + git push origin HEAD (to main or current branch)

Allowed Frontend Files

Only 5 files may be modified: src/style.css (Tailwind @theme tokens, colour palette), src/components/Navbar.vue (top navigation, colours), src/views/Home.vue (hero + layout A/B), src/components/Footer.vue (footer colours, links), frontend/index.html (OG/Twitter meta, title).

RebrandCtx (Shared Context)

repo_root	Absolute path to the myblogs repository
force_rebrand	Bypasses schedule check when True (FORCE_REBRAND=true)
chosen_theme	Theme string from IdeationAgent research (e.g. 'Ocean & Resilience')
mood	Emotional tone for the colour palette (e.g. 'calm, grounded, optimistic')
world_events	Raw text summary from research_world_events()
rebrand_plan	Dict with full colour/typography/patch plan for both layouts
files_changed	List of files patched by CodingAgent
review_cycles	Counter — capped at 3 to prevent infinite review loops
review_feedback	Issue list from ReviewerAgent (contrast or code)
test_feedback	Error output from TesterAgent

Key Files

File	Purpose
rebranding_agent/main.py	Entry point; model selection; asyncio runner; tracer
rebranding_agent/pipeline.py	build_pipeline() — constructs all 5 agents and wires handoffs
rebranding_agent/context.py	RebrandCtx dataclass

File	Purpose
rebranding_agent/tools/planning.py	check_schedule, research_world_events, generate_rebrand_plan, revise_rebrand_plan
rebranding_agent/tools/coding.py	patch_frontend_files, read_frontend_file, write_frontend_file
rebranding_agent/tools/review.py	review_seo_ada
rebranding_agent/tools/testing.py	verify_build, run_structural_tests
rebranding_agent/tools/publishing.py	commit_and_push, revert_changes
rebranding_agent/tracer.py	AgentRun lifecycle tracking